

SOFTWARE COMPOSITION ANALYSIS (SCA) DEMONSTRATION ON A SAMPLE PROJECT USING FOSSA

1. Aim

To demonstrate Software Composition Analysis (SCA) on a sample project using the FOSSA tool, in order to identify open-source dependencies, known vulnerabilities and license compliance issues.

2. Introduction

Software Composition Analysis (SCA) is the process of automatically identifying the open-source and third-party components used in an application, and checking them for known security vulnerabilities (CVEs), license obligations and outdated versions. Modern applications are largely built from open-source libraries, so a single vulnerable dependency can expose the whole application.

FOSSA is an SCA platform that scans a project's dependency manifests, builds a complete dependency tree (direct and transitive), and reports vulnerability and license-policy issues in its web dashboard. It is used through the FOSSA CLI and can also be integrated into CI/CD pipelines.

3. Requirements

- A computer with internet access (Linux, macOS or Windows)
- Node.js and npm (for the sample project)
- Git (optional)
- A FOSSA account (free trial) at <https://fossa.com>
- FOSSA CLI

4. Obtaining the FOSSA API Key

- Visit <https://app.fossa.com> and sign up or log in.
- Click your profile icon and open Account Settings.
- Select the Integrations tab and then the API Tokens section.
- Click Add Token, give it a name (for example, sca-demo) and generate it.
- Copy the token immediately and store it safely. Do not share it or commit it to a repository.

5. Installing the FOSSA CLI

On Linux or macOS, run the installer script:

```
curl -H 'Cache-Control: no-cache' https://raw.githubusercontent.com/fossas/fossa-cli/master/install-latest.sh | bash
```

Verify the installation:

```
fossa --version
```

Set the API key as an environment variable (Linux/macOS):

```
export FOSSA_API_KEY=<your_api_key_here>
```

On Windows PowerShell:

```
$env:FOSSA_API_KEY="<your_api_key_here>"
```

6. Creating the Sample Project

A small Node.js project is created that deliberately uses older library versions so that the scanner has something to report.

```
mkdir sca-demo  
cd sca-demo  
npm init -y
```

Edit package.json and add the following dependencies:

```
"dependencies": {  
  "express": "4.16.0",  
  "lodash": "4.17.15",  
  "moment": "2.19.0"  
}
```

Install the dependencies so that the lock file and dependency tree are generated:

```
npm install
```

7. Running the SCA Scan

From the project root folder, run the analysis. The CLI detects package.json and package-lock.json, resolves the full dependency tree and uploads it to FOSSA:

```
fossa analyze
```

After the analysis completes, run the policy check. This command returns a non-zero exit code if issues violating the configured policy are found, which makes it suitable for failing a CI build:

```
fossa test
```

8. Viewing and Understanding the Results

Log in to the FOSSA dashboard and open the project (named after the project or repository). The following sections are reviewed:

- **Dependencies:** list of all direct and transitive packages with their versions.

- **Security issues:** known vulnerabilities (CVE IDs), their severity (Low, Medium, High, Critical) and the fixed version, if available.
- **Licensing issues:** the license of each package (MIT, Apache-2.0, GPL and so on) and any conflicts with the license policy.
- **Outdated dependencies:** packages for which a newer version exists.

Because the sample project uses older versions of Express, Lodash and Moment, FOSSA is expected to flag known vulnerabilities in them. The exact findings depend on FOSSA's vulnerability database at the time of the scan, so record the issues that appear in your own dashboard.

Figure 1: SCA workflow with FOSSA

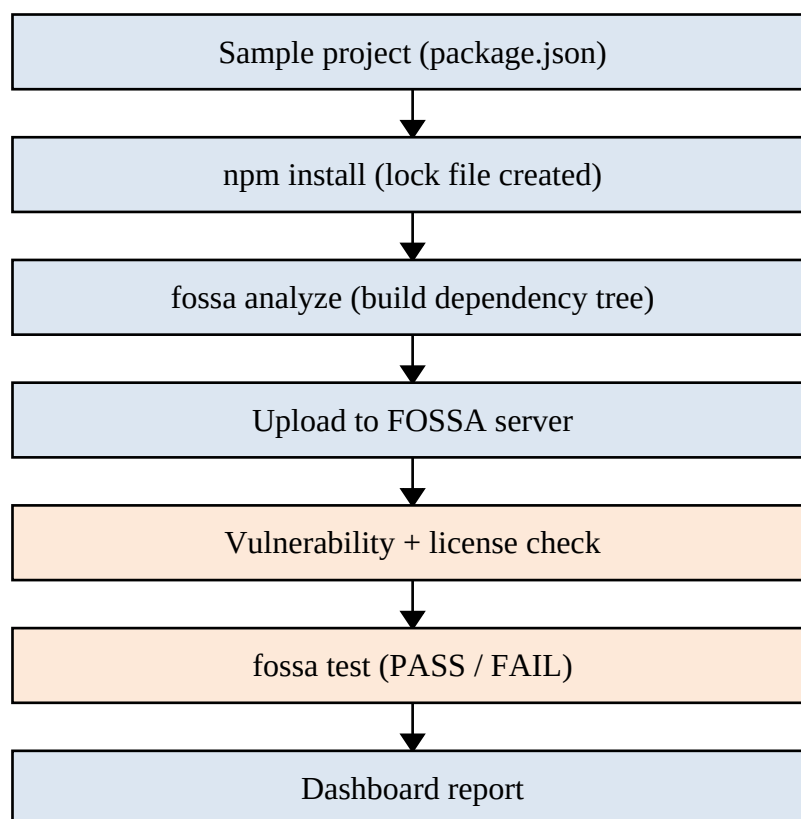
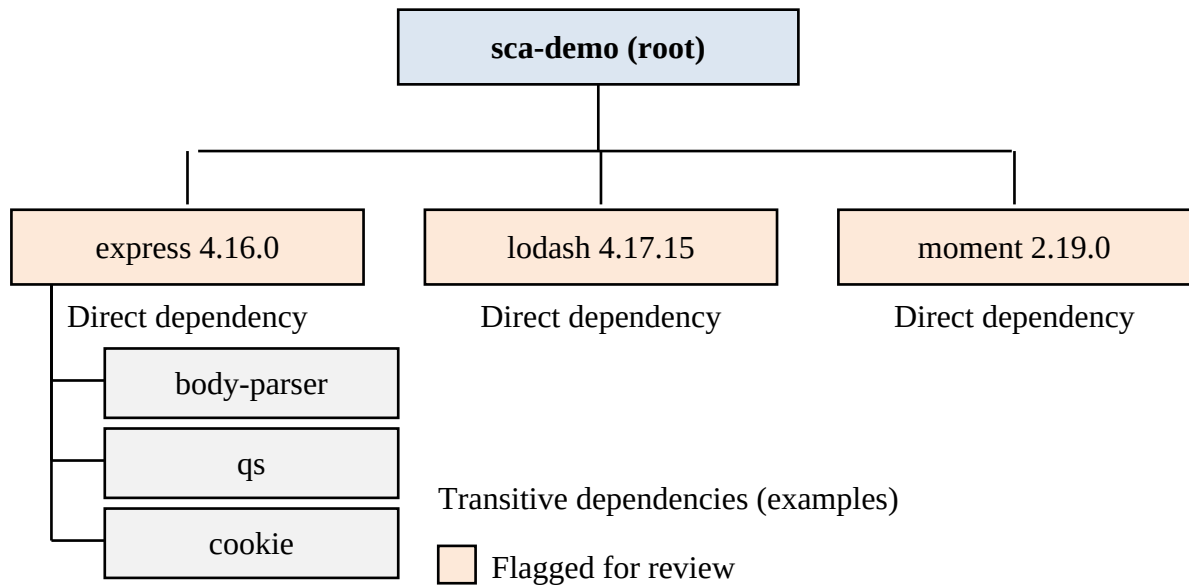
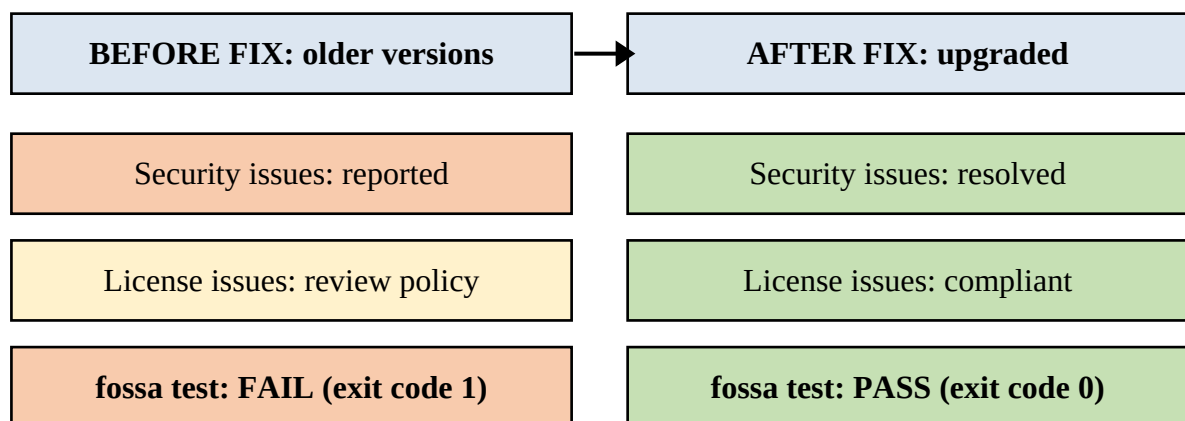


Figure 2: Dependency tree of the sample project (illustrative)**Figure 3: Scan outcome before and after remediation (illustrative)**

Note: The diagrams show the expected structure of the output. Exact vulnerability names, counts and severities come from your own FOSSA dashboard.

9. Remediation

- Upgrade each flagged package to the fixed version suggested by FOSSA.
`npm install express@latest lodash@latest moment@latest`
- Re-run *fossa analyze* and *fossa test* to confirm that the issues are resolved.
- Where an upgrade is not possible, review the issue and document an exception or mitigation.

10. CI/CD Integration (Optional)

The same commands can be added as a pipeline step (for example, in GitHub Actions) after storing the key as a secret named FOSSA_API_KEY. Running *fossa analyze* followed by *fossa test* on every commit blocks builds that introduce vulnerable or non-compliant dependencies.

11. Advantages of SCA with FOSSA

- Automatic detection of direct and transitive dependencies
- Early identification of vulnerabilities and license risks
- Policy-based enforcement and CI/CD integration
- Support for many languages and package managers

12. Result

Software Composition Analysis was successfully demonstrated on a sample Node.js project using the FOSSA tool. The project's open-source dependencies were identified, scanned for vulnerabilities and license issues, and remediated by upgrading the affected components.